

ui.timeline 全面详解

ui.timeline 是 NiceGUI 基于 Quasar 的 QTimeline 组件封装的时间线元素，用于按时间顺序展示一系列事件、里程碑或流程节点，支持自定义布局、位置、颜色等样式，广泛适用于项目进度展示、版本更新记录、历史事件梳理等场景。其核心特性包括简洁的结构组织、灵活的样式配置和丰富的元素管理能力，可快速实现直观的时间线可视化效果。

一、核心概念与基础特性

1. 本质与用途

- 本质：封装 Quasar 的 QTimeline 组件，通过 `ui.timeline_entry` 子元素定义单个时间节点，每个节点可包含标题、副标题、描述文本和图标，形成有序的时间序列展示。
- 核心用途：将离散的事件按时间先后顺序串联，清晰呈现事件的发展脉络或流程的推进步骤，提升信息的可读性与逻辑性。
- 核心组成：时间线容器（`ui.timeline()`）+ 时间节点（`ui.timeline_entry()`），每个节点支持多维度信息（标题、副标题、描述、图标），结构轻量化且易于扩展。

2. 基础结构

一个完整的 ui.timeline 由容器和多个时间节点组成，每个节点可配置丰富的信息，示例结构如下：

```
from nicegui import ui

# 时间线容器
with ui.timeline(side='right', layout='comfortable', color='blue'):
    # 时间节点1: 包含标题、副标题和描述
    ui.timeline_entry(
        '事件描述: 团队启动项目调研与技术选型',
        title='项目启动',
        subtitle='2024-01-10'
    )
    # 时间节点2: 带自定义图标
    ui.timeline_entry(
        '事件描述: 完成核心功能开发与内部测试',
        title='核心开发完成',
        subtitle='2024-03-15',
        icon='code' # Quasar 图标名称
    )
    # 时间节点3: 长文本描述
    ui.timeline_entry(
        '事件描述: 经过多轮用户反馈迭代, 修复已知问题, 优化性能与交互体验, 准备正式发布',
        title='版本发布准备',
        subtitle='2024-05-20',
        icon='check_circle'
    )

ui.run()
```

二、初始化配置项

初始化 `ui.timeline()` 时可通过参数配置整体样式与布局，参数说明如下：

参数名	类型	说明
side	str (可选值: "left" / "right")	时间节点的排列位置 (默认值为 "left", 即节点在时间线左侧; 设为 "right" 时在右侧)
layout	str (可选值: "dense" / "comfortable" / "loose")	时间线的布局密度 (默认 "dense" 紧凑布局; "comfortable" 中等间距; "loose" 宽松间距)
color	str	时间节点图标的颜色 (支持 Quasar 颜色名称如 "red"、"blue", 或十六进制颜色如 "#FF5733")

配置示例

```
# 右侧排列、宽松布局、绿色图标的 timelines
with ui.timeline(side='right', layout='loose', color='#2ECC71'):
    ui.timeline_entry('第一阶段任务完成', title='阶段1', subtitle='2024-06-01', icon='task')
    ui.timeline_entry('第二阶段任务完成', title='阶段2', subtitle='2024-07-01', icon='task')
```

三、核心属性

`ui.timeline` 继承 NiceGUI 基础元素的通用属性，支持样式、类名、可见性绑定等配置，关键属性如下：

属性名	类型	说明
classes	str	元素的 CSS 类名 (支持 Tailwind、Quasar 类, 如 <code>w-full</code> 占满宽度、 <code>mx-auto</code> 水平居中)
props	str	Quasar 组件属性 (用于扩展组件功能, 如 <code>reverse</code> 反转时间线顺序)
style	str	内联 CSS 样式 (如 <code>font-size: 16px</code> ; 调整文本大小)
visible	BindableProperty	元素可见性 (布尔值, 支持动态绑定, 如通过变量控制显示 / 隐藏)
html_id	str	HTML DOM 中的元素 ID (版本 2.16.0 新增, 用于精准定位元素)
is_deleted	bool	元素是否已被删除 (只读属性, 用于判断元素状态)
parent_slot	Slot None	父容器的插槽 (可手动设置元素所属的父插槽, 用于复杂布局嵌套)

属性使用示例

```
# 占满宽度、水平居中、蓝色边框的时间线
with ui.timeline(side='left', color='indigo').classes('w-full mx-auto border-2 border-indigo-200 p-4'):
    ui.timeline_entry('产品原型设计完成', title='原型阶段', subtitle='2024-02-01', icon='design_services')
    ui.timeline_entry('UI 视觉设计定稿', title='设计阶段', subtitle='2024-02-20', icon='palette')
```

四、核心方法

ui.timeline 提供丰富的方法用于元素管理、事件绑定、资源添加等操作，常用方法分类如下：

1. 元素管理方法

方法名	作用	参数说明
clear()	删除时间线内所有时间节点 (ui.timeline_entry)	无参数
remove(element)	删除指定的时间节点	element: 时间节点实例 (ui.timeline_entry 对象) 或其 ID
delete()	删除整个时间线元素及所有子节点	无参数
move(target_container=None, target_index=-1, target_slot=None)	移动时间线到其他容器	target_container: 目标容器 (默认父容器) ; target_index: 目标索引 (默认追加到末尾) ; target_slot: 目标插槽

2. 可见性绑定方法

支持将时间线的可见性与外部变量绑定，实现动态显示 / 隐藏，核心方法如下：

方法名	作用	关键参数
bind_visibility(target_object, target_name='visible')	双向绑定可见性到目标对象的属性	target_object: 绑定目标对象; target_name: 绑定的属性名 (默认 'visible')
bind_visibility_from(...)	单向绑定 (从目标对象同步到时间线)	同 bind_visibility, 仅单向同步 (目标对象属性变化触发时间线可见性变化)
bind_visibility_to(...)	单向绑定 (从时间线同步到目标对象)	同 bind_visibility, 仅单向同步 (时间线可见性变化触发目标对象属性变化)

3. 其他常用方法

方法名	作用	示例
tooltip(text)	为时间线添加鼠标悬浮提示	<code>timeline.tooltip('项目进度时间线')</code>
update()	强制更新客户端的时间线状态（如样式、节点变化后刷新）	<code>timeline.update()</code>
add_resource(path)	为时间线添加资源文件（如自定义 CSS、JS）	<code>timeline.add_resource('./static')</code>
ancestors(include_self=False)	迭代获取所有祖先元素	遍历祖先元素： <code>for elem in timeline.ancestors(): print(elem)</code>
mark(*markers)	为元素添加标记（用于测试或元素查询）	<code>timeline.mark('project-timeline', '2024')</code>

方法使用示例

```
from nicegui import ui

# 动态添加/删除时间节点
def add_entry():
    # 新增时间节点并添加到时间线末尾
    entry = ui.timeline_entry(
        '新增临时任务完成',
        title='临时任务',
        subtitle='2024-08-01',
        icon='add_task'
    )
    entries.append(entry) # 保存节点引用，用于后续删除

def delete_last_entry():
    if entries:
        timeline.remove(entries.pop()) # 删除最后一个节点
        timeline.update() # 刷新时间线

entries = []
with ui.timeline(side='left', color='teal') as timeline:
    ui.timeline_entry('初始任务1完成', title='任务1', subtitle='2024-07-01', icon='task')
    ui.timeline_entry('初始任务2完成', title='任务2', subtitle='2024-07-15', icon='task')

# 操作按钮
ui.button('添加节点', on_click=add_entry).props('color=teal')
ui.button('删除最后一个节点', on_click=delete_last_entry).props('color=red')

ui.run()
```

五、时间节点 (ui.timeline_entry) 配置

`ui.timeline_entry` 是时间线的核心子元素，用于定义单个时间节点的内容与样式，其参数说明如下：

参数名	类型	说明
第一个参数	str	节点的描述文本（必填，支持多行字符串）
title	str	节点的标题（可选，通常为事件名称或阶段名称，字体加粗显示）
subtitle	str	节点的副标题（可选，通常为时间、地点等补充信息，字体较小）
icon	str	节点的图标（可选，支持 Quasar 图标库中的图标名称，如 'rocket'、'check'）

节点配置示例

```
# 带图标、标题、副标题和长描述的时间节点
ui.timeline_entry(
    '通过社区反馈收集到100+有效建议，针对核心功能进行优化，同时修复3个高优先级bug，'
    '优化后系统响应速度提升30%，用户满意度显著提高',
    title='v2.0 版本优化',
    subtitle='2024-09-30',
    icon='build'
)
```

六、事件处理

`ui.timeline` 支持绑定通用 DOM 事件（如点击、鼠标悬浮），通过 `on()` 方法实现事件响应，支持 Python 回调或 JavaScript 回调：

1. 绑定 Python 回调（服务端处理）

```
def on_timeline_click(e):
    ui.notify('点击了时间线容器')

# 为时间线容器绑定点击事件
with ui.timeline(side='left') as timeline:
    timeline.on('click', on_timeline_click)
    ui.timeline_entry('事件1', title='标题1', subtitle='2024-01-01')
```

2. 绑定 JavaScript 回调（客户端处理）

```
# 为时间线容器绑定鼠标悬浮事件，客户端打印日志
with ui.timeline(side='left') as timeline:
    timeline.on('mouseover', js_handler='(e) => console.log("鼠标悬浮时间线：", e)')
    ui.timeline_entry('事件2', title='标题2', subtitle='2024-01-02')
```

3. 为单个时间节点绑定事件

时间节点（`ui.timeline_entry`）作为独立元素，也支持事件绑定：

```
entry = ui.timeline_entry('可点击的节点', title='交互节点', subtitle='2024-01-03',
    icon='hand_point_up')
entry.on('click', lambda: ui.notify('点击了"交互节点"'))
```

七、高级用法

1. 样式深度自定义

结合 `classes`、`style` 和 Quasar props 实现个性化样式，示例：

```
# 反转时间线（最新事件在前）、红色图标、自定义节点间距
with ui.timeline(side='right', layout='comfortable',
    color='red').props('reverse').style('gap: 2rem;'):
    ui.timeline_entry('最新事件：产品正式上线', title='上线发布', subtitle='2024-10-15',
        icon='launch')
    ui.timeline_entry('上线前最终测试', title='测试阶段', subtitle='2024-10-10',
        icon='verified_user')
    ui.timeline_entry('上线准备工作完成', title='准备阶段', subtitle='2024-10-05',
        icon='done_all')
```

2. 动态控制时间线可见性

通过绑定变量控制时间线显示 / 隐藏，适用于条件展示场景：

```
from nicegui import ui

class AppState:
    show_timeline = True

state = AppState()

# 双向绑定：勾选框控制时间线可见性
ui.checkbox('显示时间线', value=state.show_timeline).bind_value(state, 'show_timeline')

with ui.timeline(side='left') as timeline:
    timeline.bind_visibility(state, 'show_timeline') # 绑定可见性
    ui.timeline_entry('事件A', title='标题A', subtitle='2024-01-01')
    ui.timeline_entry('事件B', title='标题B', subtitle='2024-01-02')

ui.run()
```

3. 嵌套复杂元素

时间节点的描述文本支持嵌套其他 NiceGUI 元素（如按钮、链接、图片），实现交互增强：

```
with ui.timeline(side='left'):  
    # 描述中嵌套按钮和链接  
    with ui.timeline_entry(title='重要里程碑', subtitle='2024-04-01', icon='star'):  
        ui.label('里程碑达成, 点击查看详情: ')  
        ui.button('查看报告', on_click=lambda: ui.notify('报告已打开')).props('color=blue')  
        ui.link('访问项目官网', 'https://example.com').props('color=green')  
    # 描述中嵌套图片  
    ui.timeline_entry(  
        ui.image('https://picsum.photos/200/100').classes('mt-2'), # 图片元素  
        title='项目截图',  
        subtitle='2024-04-10',  
        icon='image'  
    )
```

八、注意事项

- 图标兼容性: `icon` 参数仅支持 Quasar 图标库中的名称（如 'rocket'、'check_circle'），需确保图标名称正确，否则将显示默认占位图标。
- 颜色配置: `color` 参数支持 Quasar 预定义颜色（如 'primary'、'secondary'）、英文颜色名（如 'orange'）和十六进制颜色码（如 '#FFA500'），不支持 RGB 格式。
- 布局密度: `layout` 参数的三个值对应不同间距，需根据节点数量和页面布局选择，避免 "loose" 布局在节点过多时占用过多空间。
- 版本兼容性: `html_id` 属性需 NiceGUI 2.16.0+ 版本支持，`bind_visibility` 的 `strict` 参数需 3.0.0+ 版本支持，使用时需确认版本匹配。
- 动态节点管理: 通过 `clear()`、`remove()` 方法删除节点后，建议调用 `update()` 方法刷新客户端显示，确保界面状态同步。

通过以上配置与方法，`ui.timeline` 可灵活满足从简单事件记录到复杂交互时间线的各类需求，是 NiceGUI 中实现时间序列信息可视化的核心组件之一。